cmmdstack

The skill library every agent in a CMMD repo uses, on Claude Code, Codex and Cursor. Give the agent a goal and a check. It picks the playbook and the skills.

C/

Start every real task here

Type `/cmmd-mode` when the work touches more than one file, makes a design choice, or chases a bug with an unknown cause. It matches a playbook, copies its steps into the todo list, and calls the other skills as the steps need them.

```
/cmmd-mode Saving a task twice creates two rows.
Done when the e2e spec shows one row and a lint
blocks the pattern.
```

Two questions before you start

`/ask-alex` What would Alex do here? Names the flow and the skills, and hands back a prompt to send.

`/ask-zack` Is this worth doing now? Checks the work against the strategy in CMMD Brain. An **OFF TRACK** verdict stops the work until the owner overrides it.

The hard rules

1. **Every bug ends in a guard.** Fix it, then `/correct` adds a lint, type check or static check that fails on the bad code, in the same change. A test only when no static check can say it.
2. **Done means proven on the real surface.** The browser, the API, the CLI, an end-to-end run. Green unit tests are not proof.
3. **Review comes from another model.** Never the author, never your own read of your own diff.
4. **Facts come from running something.** A question a check can settle never goes to a human.
5. **AGENTS.md is the one instruction file.** CLAUDE.md is `@AGENTS.md`. The build fails otherwise.
6. **Plain prose.** `/unslop` on every PR body, doc and comment. No em dashes.

Idea to merged PR

01
`/debrief`
Interview until every decision is settled. Writes CONTEXT.md and ADRs.

02
`/architect`
Types and module shape before code that crosses files.

03
`/cmmd-mode`
Feature playbook. Test first with `/tdd`.

04
`/interrogate`
Multi-model review of the diff. `/council` for three providers.

05
`/file-pr`
Then `/babysit-pr` until green and reviewed.

Something is broken

- `/cmmd-mode` Bug fix: reproduce, root cause, fix, guard
- `/why` when it used to work
- `/how` to map the subsystem first
- `/fix-ci` for a red pipeline

Lots of similar work

- `/swarm` N workers, one report
- `/arena` race designs, keep the best parts
- `/figure-it-out` big migration, own playbook
- `/research` primary sources, cited note

Walking away

- `/full-stop` runs to merge-ready, never merges
- `/full-send` runs through the merge
- `/show-me-your-work` leaves a decision trail
- `/ketchup` catches you up after

A prompt that works

State the goal and a check that can pass or fail. Leave out the skill list, because `/cmmd-mode` picks the skills, and a hand-written list drops steps.

Weak

```
Use tdd and swarm to make the board faster.
```

Strong

```
Board loads in under 800 ms on the 2k-card fixture.
Prove it with the perf spec before and after.
```

Which skill, when

C/

SITUATION	SKILL
New idea, plan not clear yet	`/debrief`
Not sure it is worth doing	`/ask-zack`
Not sure which skill fits	`/ask-alex`
Bug with an unknown cause	`/cmmd-mode`
Same mistake made twice	`/correct`
Design that crosses files	`/architect`
Contested design or diff	`/interrogate`
Several candidate fixes	`/arena`
Sweep, race, coverage matrix	`/swarm`
How does X work	`/how`
Why was Y built this way	`/why`
Docs or API facts	`/research`
What breaks if I change this	`/blast-radius`
Large migration	`/figure-it-out`
State machine or race	`/statechart`

SITUATION	SKILL
Prove UI, no driver yet	`/create-verification-skill`
Invariant tests miss	`/create-formal-verification-skill`
Report a speedup	`/benchmark-checklist`
Before a commit	`/deslop`
Before review	`/no-comments`
Open a PR	`/file-pr` or `/pr`
Watch a PR to green	`/babysit-pr`
Bot review comments	`/get-pr-comments`
Merge conflicts	`/fix-merge-conflicts`
Take over a PR	`/takeover`
Old PR, new main	`/modernize`
GPT review loop	`/sol-loop`
PR body or doc	`/unslop`
What did I ship	`/what-did-i-get-done`
Find an old chat	`/recall`

Playbooks inside /cmmd-mode

Name the situation and the mode matches one.

Investigation · Bug fix · Perf issue · Hillclimb · Runtime forensics · Trace forensics · Feature · Refactoring · Prototype · Visual parity · Authoring a skill · Eval · Babysit · Shipping · Autonomous run · Orchestrate · Autopilot · Session pickup · Pause safely · Multi-phase plan · Worktree cleanup · Opening a PR

Principles it cites

Each is its own `principle-*` skill.

Laziness protocol · Subtract before you add · Attack the premise · Model the domain · Boundary discipline · Type system discipline · Make operations idempotent · Prove it works · Fix root causes · Test behavior, not implementation · Explain the number · Build the lever · Guard the context window · Never block on the human · Encode lessons in structure

Install and update

In CMMD, `node scripts/install-cmmdstack.mjs` installs the latest tag into `.agents/skills/` and mirrors it for every tool.

`npm run check:cmmdstack-fresh` says whether a newer tag exists. A weekly job opens the update PR for you.

Outside CMMD: `/plugin marketplace add CMMD-Center/cmmdstack`

Changing a skill

- Edit it in the cmmdstack repo, run `node scripts/check.mjs`, release a tag.
- A hand edit to `.agents/skills/` in CMMD fails `check:skills-vendored`.
- CMMD-only skills (Sidekick, Forge, migrations) stay unpinned in CMMD.
- The rules in `.agents/` win when a skill disagrees.